

INFORME TÉCNICO

EVIDENCIA DE APRENDIZAJE 4

Proyecto: Web Scraping, enriquecimiento y visualización de metadatos de paquetes
Python en PyPI

Campo	Detalle
Estudiantes	Jonatan Ávila y Estiven Trujillo
Programa	Análisis de Datos
Metodología	CRISP-DM
Herramientas	Python, BeautifulSoup, Requests, MySQL, Power BI, GitHub Actions
Fecha	Medellín, junio de 2026

1. Introducción

Este informe presenta la consolidación de la Evidencia de Aprendizaje 4, orientada a comunicar el desarrollo completo de un proyecto de análisis de datos aplicado al ecosistema PyPI. El proyecto parte de la extracción de metadatos públicos de paquetes Python, continúa con la transformación y enriquecimiento de la información en una base de datos relacional y finaliza con la visualización de indicadores en Power BI.

La entrega integra el producto técnico y la comunicación profesional del trabajo realizado: código del scraper, base de datos enriquecida, tablero de Power BI, versionamiento en Git, configuración de integración continua y una narrativa clara para el video de sustentación.

2. Descripción del problema

PyPI concentra una gran cantidad de librerías utilizadas en proyectos de software, ciencia de datos, automatización, seguridad, pruebas y desarrollo web. Sin embargo, elegir una dependencia solo con base en su nombre o descripción puede aumentar riesgos técnicos: paquetes poco mantenidos, licencias ambiguas, baja compatibilidad, dependencias excesivas o falta de trazabilidad del repositorio.

El problema abordado consiste en convertir metadatos públicos dispersos en indicadores comparables que apoyen la toma de decisiones sobre madurez, complejidad y utilidad de paquetes Python. La solución permite observar patrones por categoría y priorizar librerías con mayor estabilidad para proyectos reales.

3. Objetivos

- Construir un flujo de extracción de datos desde PyPI usando Python.
- Almacenar la información en una base de datos MySQL con estructura relacional.
- Enriquecer los datos con variables derivadas: madurez, complejidad de dependencias, antigüedad, licencias normalizadas y soporte de Python 3.
- Visualizar los resultados en Power BI para facilitar el análisis de categorías, paquetes y riesgos.
- Documentar el proceso bajo la metodología CRISP-DM e incorporar buenas practicas de versionamiento e integración continua.

4. Datos disponibles

La base de datos final, llamada `pypi_catalog`, contiene 42 paquetes organizados en 8 categorías: Data Science, Web Scraping, Web Framework, Security, Testing, Database, Utilities y Networking. La tabla principal `paquetes` integra metadatos originales y campos enriquecidos.

Elemento	Descripción
categorías	Catalogo de categorías funcionales para clasificar los paquetes.
paquetes	Tabla principal con nombre, versión, resumen, licencia, dependencias, releases, categoría y campos enriquecidos.
Campos técnicos	versión_actual, requiere_python, url_repositorio, total_releases, n_dependencias y fecha_primera_release.
Campos enriquecidos	anios_desde_primera_versión, nivel_madurez, complejidad_dependencias, tiene_github y soporta_python3.

5. Solución propuesta

5.1 implementación del scraper

El scraper fue implementado en Python utilizando requests para consultar páginas públicas de PyPI y BeautifulSoup para interpretar el HTML. La función principal `get_pypi_data(package_name)` recibe el nombre de un paquete, consulta su página pública y extrae atributos base como nombre, resumen y versión actual.

Mejora técnica incorporada

El archivo `scraper.py` fue ajustado para poder importarse sin ejecutar el proceso completo, usar variables de entorno para la conexión a MySQL y manejar respuestas no exitosas o páginas con estructura incompleta.

La carga se integra con MySQL a través de SQLAlchemy y PyMySQL. El dump `pypi_catalog_completo (1).sql` conserva la estructura completa de la base y los registros enriquecidos usados por Power BI.

5.2 Transformación y enriquecimiento

- Nivel de madurez: clasifica los paquetes como Maduro o Estable según antigüedad e historial de versiones.
- Complejidad de dependencias: agrupa el número de dependencias en Baja, Media o Alta para evaluar esfuerzo de integración.
- Antigüedad: calcula los años desde la primera versión registrada.
- Licencia normalizada: estandariza valores como MIT, BSD, GPL, Apache-2.0 o Sin especificar.
- Indicadores binarios: identifica soporte para Python 3 y presencia de repositorio asociado.

6. Metodología CRISP-DM

1. Comprensión del negocio: se definió la necesidad de evaluar paquetes Python desde criterios de madurez, compatibilidad y riesgo de integración.
2. Comprensión de los datos: se identificaron las fuentes públicas de PyPI y los campos disponibles en las páginas de cada paquete.

3. Preparación de los datos: se limpiaron textos, se normalizaron licencias, se calcularon campos derivados y se estructuró la base MySQL.
4. Modelado: se diseñó un modelo relacional simple entre categorías y paquetes, suficiente para filtros y agregaciones en Power BI.
5. Evaluación: se revisó la consistencia de categorías, conteos, soporte Python 3, distribución de madurez y dependencias.
6. Despliegue: se conectó la base enriquecida a Power BI y se generó el tablero de análisis para la sustentación.

7. Análisis de datos enriquecidos

Indicador	Resultado
Total de paquetes	42
Categorías	8
Madurez	33 paquetes Maduros y 9 Estables
Complejidad	14 Baja, 14 Media y 14 Alta
Soporte Python 3	42 de 42 paquetes
Repositorios asociados	39 de 42 paquetes
Promedio de dependencias	17.76 dependencias por paquete
Mayor número de releases	hypothesis con 1516 releases
Mayor número de dependencias	Twisted con 150 dependencias

Los resultados muestran un ecosistema mayoritariamente maduro: 78.6 % de los paquetes analizados se clasifican como Maduros. La complejidad se distribuye de manera balanceada entre Baja, Media y Alta, lo que facilita comparar perfiles de integración según el tipo de proyecto.

También se observa que todos los paquetes de la muestra soportan Python 3, mientras que 39 de 42 tienen un repositorio asociado o trazabilidad externa. Estos indicadores fortalecen la lectura de confiabilidad, mantenimiento y adopción del ecosistema.

8. Conexión y visualización en Power BI

Power BI se conectó a la base MySQL pypi_catalog mediante el conector nativo. La relación principal se establece entre categorías.id y paquetes.categoría_id, permitiendo segmentar los indicadores por categoría funcional.

- Tarjetas KPI: total de paquetes, categorías, paquetes maduros y soporte Python 3.
- Graficos de barras: distribución de paquetes por categoría y complejidad de dependencias.
- Graficos de dona o columnas: licencias normalizadas y niveles de madurez.
- Tabla detallada: comparación por paquete con versión, categoría, releases, dependencias y repositorio.
- Filtros interactivos: categoría, madurez, complejidad y licencia.

9. Versiónamiento e integración continúa

El proyecto se encuentra versionado con Git y cuenta con un commit base llamado Entrega 3: Transformación y Análisis de Datos Enriquecidos. Como valor agregado, se incorporo una configuración de GitHub Actions en `.github/workflows/ci.yml` para ejecutar validaciones automáticas del proyecto.

Práctica	Implementación
Versiónamiento	Repositorio Git con historial de cambios y archivos fuente del proyecto.
Dependencias	requirements.txt con librerías necesarias para ejecutar el scraper.
Pruebas	tests/test_scrapper.py valida extracción de metadatos, manejo de errores y URL de base de datos.
CI	Workflow CI que instala dependencias, ejecuta unittest y valida sintaxis de scrapper.py.

10. Resultados y conclusiones

- El proyecto demuestra que el web scraping puede transformar información pública dispersa en una base analítica útil.
- El enriquecimiento de datos agrega valor porque permite evaluar madurez, complejidad, licencias y compatibilidad, no solo listar paquetes.
- Power BI facilita la comunicación de hallazgos mediante filtros, KPIs y comparaciones visuales.
- La metodología CRISP-DM ayudó a organizar el proyecto desde la necesidad hasta el despliegue del tablero.
- La adición de versiónamiento, pruebas e integración continúa mejora la calidad profesional de la entrega.

11. Guion recomendado para el video

Bloque	Duración	Contenido
Introducción	0:00-0:40	Presentar el proyecto PyPI Catalog, estudiantes, objetivo y contexto.
Necesidad y solución	0:40-1:40	Explicar el problema de seleccionar librerías sin indicadores comparables y la solución de scraping + enriquecimiento.
Código	1:40-3:20	Mostrar scrapper.py, get_pypi_data, conexión MySQL, SQL dump, requirements, pruebas y CI.
Power BI	3:20-5:00	Mostrar KPIs, filtros, categorías, madurez, complejidad, licencias y tabla de paquetes.
Conclusiones	5:00-6:00	Resumir hallazgos y valor de CRISP-DM, transformaciones, Power BI y buenas prácticas.

12. Bibliografia

- Beautiful Soup. (2026). Beautiful Soup documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- IBM. (2021). CRISP-DM. IBM Cloud Learn Hub. <https://www.ibm.com/cloud/learn/crisp-dm>
- Microsoft. (2026). Documentacion de Power BI. <https://learn.microsoft.com/es-es/power-bi/>
- Python Software Foundation. (2026). PyPI - The Python Package Index. <https://pypi.org/>
- Python Software Foundation. (2026). unittest - Unit testing framework. <https://docs.python.org/3/library/unittest.html>
- Requests. (2026). Requests: HTTP for Humans. <https://requests.readthedocs.io/>
- SQLAlchemy. (2026). SQLAlchemy documentation. <https://docs.sqlalchemy.org/>